BCSE307P Compiler Design Lab

Innovative Project

Assignment 6 · Project Report · Team 9

CADENCE: A Music Domain-Specific Language Compiler with a Glass-Box Studio

by

Reg No	Team Member Names
24BDS0252	Rohit Arora
24BCE0520	Anshika Singh
24BKT0088	Arshiya Verma

Under the Supervision of

Dr. Muthunagai S. U

Assistant Professor Sr. Grade-2, Department of IoT

School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore



25 September 2026

1. Abstract:

Cadence is a domain-specific language for writing music and a complete compiler that translates it into a Standard MIDI File. A Cadence program declares a tempo, a key, instruments, reusable phrases with integer parameters and parallel voices; the compiler produces a `.mid` file that any media player or keyboard can play. The compiler is written from scratch in pure Python with no third-party package, not even a MIDI library: every phase of the course is implemented from its algorithm and every intermediate artefact can be inspected.

The front end turns the token specification, written as extended regular expressions, into one minimised DFA by the direct method and scans with maximal munch. A single grammar of 56 productions is compiled into SLR(1), CLR(1) and LALR(1) tables (114, 204 and 114 states), the LALR(1) table drives the parser with panic-mode recovery, and a hand-written recursive-descent parser cross-checks every tree. Semantic analysis is an L-attributed syntax-directed definition in which key, octave, duration, velocity, transposition and stretch are inherited attributes, so a scale degree such as `^5` becomes a pitch only when the enclosing key reaches it. The middle end generates three-address code with activation records and backpatched loops, verifies it by abstract interpretation, and runs thirteen optimisation passes driven by live-variable, reaching-definition and available-expression analyses. The back end is a virtual machine with a real frame stack, register allocation by graph colouring in which the registers are the sixteen MIDI channels, and a byte-level MIDI writer.

On the showcase program the optimiser cuts the instructions executed from 631 to 241 (61.8% less work) while a differential re-execution after every pass proves the music unchanged. On top of the compiler the project adds Cadence Studio, a piano-centric web workstation served by FastAPI and uvicorn: an 88-key playable piano, a score editor that compiles as you type, a piano roll with browser-side audio, ten mood presets, a Glass Box that renders every compiler phase, and an AI composer whose every suggestion is compiled before it is shown. The whole system is covered by 97 automated tests.

Keywords: domain-specific language, lexical analysis, DFA, LALR(1), syntax-directed translation, inherited attributes, three-address code, data-flow analysis, register allocation, MIDI, FastAPI, Web Audio.

2. Introduction:

2.1 Background and motivation

A compiler reads a text written for people, works out exactly what was meant, checks that it makes sense, quietly improves it, and emits instructions for a machine. The Compiler Design syllabus walks through that pipeline module by module: regular expressions and finite automata, context-free grammars and LR parsing, syntax-directed definitions, intermediate code, optimisation and data-flow analysis, and code generation with run-time organisation. The usual difficulty for a course project is that a toy language gives each module nothing real to do: there are no inherited attributes worth having, nothing worth optimising, and no target that anyone can observe.

Music turned out to be an unusually good target domain. A note written as a scale degree has no pitch until the key reaches it, which makes inherited attributes a necessity rather than an exercise. Repetition and transposition compile into loops and arithmetic that an optimiser can fold, unroll and inline. Voices that play at the same time must be placed on a fixed set of MIDI channels, which is register allocation with sixteen registers and one of them reserved. Above all, the output is audible: whether an optimisation pass is correct can be settled by checking that the music did not change.

2.2 What Cadence is

Cadence is that language and that compiler. The complete program below declares two instruments, a parameterised phrase, two voices and plays them together. Everything after it in this report is what happens to these fourteen lines on their way to a `.mid` file.

Listing 1. examples/03_riff.cad, the program used throughout this report

```
// Phrases with parameters, chords, repetition and transposition.
// This is the program used in the report and the slides.


tempo 120 ;
key D minor ;
duration 8 ;
velocity 96 ;


instrument piano = "grand piano" ;
instrument bass  = "electric bass" ;


// A phrase is a procedure.  Calling it pushes a real activation record.
phrase riff(t) = transpose t of [ D4 F4 A4 <D4 F4 A4>/4 ] ;
phrase tail    = [ ^5 ^3 ^1/4 r/8 ] ;
```

```
voice lead on piano = repeat 2 of riff(0) tail ;
voice low  on bass  = transpose -12 of [ repeat 2 of riff(0) ] ;


play lead | low ;             // '|' means "at the same time"
```

Table 1. *The whole language*

You write	It means
`tempo 120 ;`	beats per minute
`key D minor ;`	the key, which is what makes `^5` mean something
`duration 8 ;`	notes with no length written are eighth notes
`velocity 96 ;`	how hard notes are struck (1 to 127)
`instrument p = "grand piano" ;`	by General MIDI name, by number 0 to 127, or `drums`
`C4  D#4  Bb3  F4/16  G4/4.`	notes; `/16` is a length, a trailing dot dots it
`^1 ^5 ^3'  ^2,`	scale degrees; an apostrophe is up an octave, a comma down
`r/8` or `~/8`	a rest
`<C4 E4 G4>/4`	a chord
`[ ... ]`	a group
`repeat 4 of X`	repetition
`transpose -12 of X`	shift by semitones
`slow 2 of X`, `fast 2 of X`	stretch or squeeze the note lengths
`phrase riff(t) = ... ;`	a reusable phrase, with parameters
`voice lead on p = ... ;`	a part, assigned to an instrument
`play lead | low ;`	play these together

2.3 Objectives

- Implement every phase of the syllabus from its algorithm, in pure Python, with no parser generator, no regular-expression library and no MIDI library.
- Make every intermediate artefact observable: tokens, DFA, parse tables, parse trace, syntax tree, symbol table, attribute flow, three-address code before and after each pass, data-flow fixed points, verifier report, activation records, channel allocation and the bytes of the output.
- Measure rather than claim: table sizes, conflict counts, instruction counts before and after optimisation, and a per-pass proof that the music is unchanged.
- Provide a professional, piano-centric user interface on top of the compiler so that the system can be used to make music as well as to study compilation.
- Cover the whole system with automated tests, including differential tests between independent implementations.

2.4 Organisation of this report

Section 3 states what is novel. Section 4 describes the design of the language, the compiler and the Studio. Section 5 presents the implementation phase by phase with code excerpts, measurements and screenshots of the running software. Section 6 concludes, and Section 7 gives the repository.

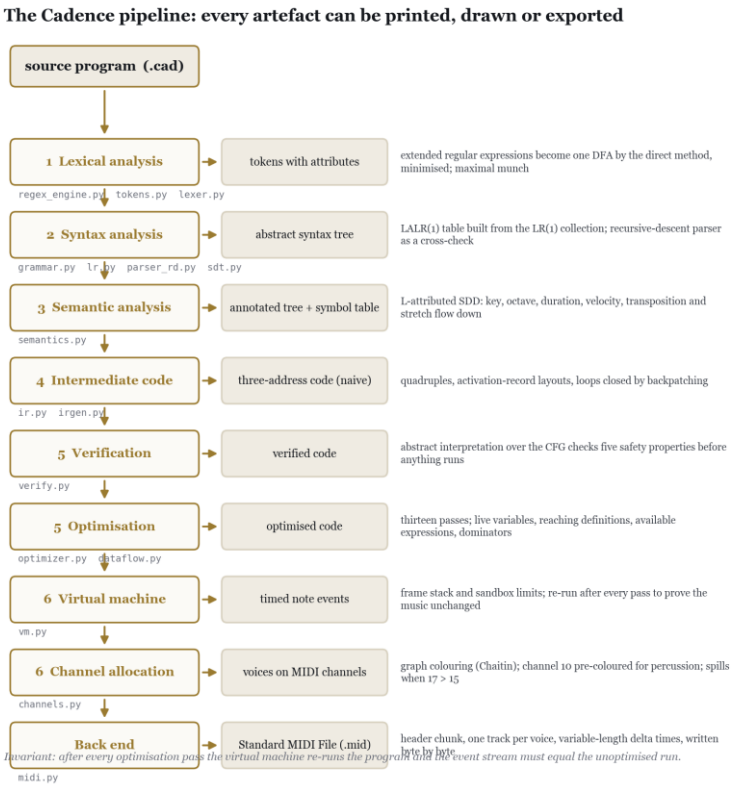


Figure 1. *The twelve-stage pipeline and the artefact each stage produces.*

3. Novelty in Project:

The project is novel in four ways, each of which can be checked in the code and in the measurements of Section 5.

3.1 Every module does real work in one artefact

The scanner is built by the direct method from extended regular expressions to a DFA, then minimised by partition refinement; Python's `re` module is never imported. One grammar is compiled into LL(1), SLR(1), CLR(1) and LALR(1) tables so that the trade-off between them can be measured on the project's own language, and the deliberately ambiguous first draft of the grammar is kept so that the conflict diagnoser can be shown finding its 24 shift/reduce and 2 reduce/reduce conflicts. The semantic analyser is a genuine L-attributed definition with six inherited and three synthesised attributes. The optimiser runs thirteen passes over quadruples and the data-flow analyses record every iteration of their fixed points. The code generator performs register allocation by Chaitin's graph colouring with a pre-coloured node and spilling.

3.2 Optimisation that must be inaudible

After every single optimisation pass the virtual machine re-runs the program and the note-by-note event stream is compared with the unoptimised run. A pass that changes what you hear fails the build. This turns the usual, unverifiable claim that an optimiser is correct into a property that the test suite checks for every example program at every optimisation level, on every run.

3.3 Security checking of virtual-machine code

The syllabus names security checking of virtual machine code, and the project takes it seriously: before the VM executes anything, the generated code is checked by abstract interpretation over its control-flow graph for the same five families of property that a JVM bytecode verifier, a WebAssembly validator or the eBPF verifier checks, and the VM itself is sandboxed by step, frame-depth and event limits.

3.4 A glass box, and a studio built on it

Nothing inside the compiler is hidden. The command line can dump any phase, a Tkinter window shows one tab per phase, a self-contained HTML report reproduces every table, and Cadence Studio renders all of it in a browser next to a playable piano. The Studio adds something a compiler project rarely has: a creative front end. Music

played on the keyboard is written back as Cadence tokens, either as absolute notes or as scale degrees of the compiled key; the compiled event stream is played through the browser's audio engine with the keys lighting up; and an AI composer backed by Groq proposes whole programs that the compiler verifies, and repairs through the compiler's own diagnostics, before the user ever sees them. The compiler is thus used as a guard-rail for a language model, which is a contemporary use of exactly the diagnostics the course asks for.

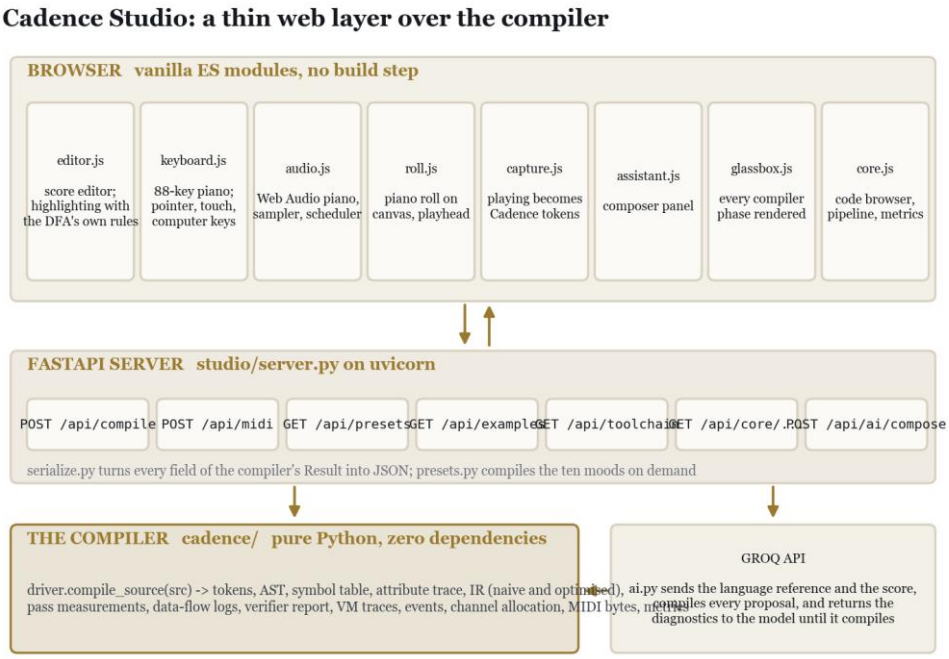


Figure 2. *Cadence Studio is a thin web layer over the unchanged compiler.*

4. Proposed Work:

4.1 The language

A program is a sequence of declarations followed by exactly one `play` statement. Declarations set the tempo, key, default duration and velocity, name instruments, and define phrases and voices. Music is built from notes, scale degrees, rests, chords and groups, combined by sequencing, by the parallel operator `|`, and by the operators `repeat`, `transpose`, `slow` and `fast`, each of which applies to one element (a group when several are needed). Phrases are procedures: they may take integer parameters, which can be used only in expressions, and they may not be recursive. Only voices may appear in `play`.

Table 2. *The token specification: 25 rules; 19 reserved words are folded in through a keyword table*

Token	Extended regular expression (priority order)
WS	`[ \t\r\n]+`
COMMENT	`//[^\n]*`
STRING	`\"[^\"\n]*\"`
BADSTRING	`\"[^\"\n]*`
NOTE	`[A-G](#|b)?[0-8](/(1|2|4|8|16|32|64)\.?)?`
DEGREE	`\^[1-7]('|,)*(/(1|2|4|8|16|32|64)\.?)?`
REST	`(r|~)(/(1|2|4|8|16|32|64)\.?)?`
PITCHNAME	`[A-G](#|b)?`
NUMBER	`[0-9]+`
ID	`[a-z_][a-zA-Z0-9_]*`
ASSIGN	`=`
SEMI	`;`
BAR	`|`
LBRACK	`\[`
RBRACK	`\]`
LPAREN	`\(`
RPAREN	`\)`
SLASH	`/`

Token	Extended regular expression (priority order)
DOT	`\.`
PLUS	`\+`
MINUS	`-`
STAR	`\*`
COMMA	`,`
LT	`<`
GT	`>`

Table 3. *The grammar: 56 productions, left-recursive by design and parsed by LALR(1)*

#	Production	#	Production
0	`program -> decls play`	28	`elem -> FAST expr OF elem`
1	`decls -> decls decl`	29	`elem -> LBRACK comp RBRACK`
2	`decls -> ε`	30	`elem -> LT notelist GT SLASH dur`
3	`decl -> TEMPO expr SEMI`	31	`elem -> NOTE`
4	`decl -> KEY PITCHNAME MODE SEMI`	32	`elem -> DEGREE`
5	`decl -> VELOCITY expr SEMI`	33	`elem -> REST`
6	`decl -> DURATION NUMBER SEMI`	34	`elem -> PITCHNAME`
7	`decl -> INSTRUMENT ID ASSIGN insref SEMI`	35	`elem -> ID`
8	`decl -> PHRASE ID params ASSIGN comp SEMI`	36	`elem -> ID LPAREN arglist RPAREN`
9	`decl -> VOICE ID onclause ASSIGN comp SEMI`	37	`notelist -> notelist noteatom`
10	`insref -> NUMBER`	38	`notelist -> noteatom`
11	`insref -> STRING`	39	`noteatom -> NOTE`
12	`insref -> DRUMS`	40	`noteatom -> DEGREE`
13	`params -> LPAREN idlist RPAREN`	41	`noteatom -> PITCHNAME`
14	`params -> ε`	42	`dur -> NUMBER`
15	`idlist -> idlist COMMA ID`	43	`dur -> NUMBER DOT`
16	`idlist -> ID`	44	`arglist -> arglist COMMA expr`
17	`onclause -> ON ID`	45	`arglist -> expr`
18	`onclause -> ON DRUMS`	46	`expr -> expr PLUS term`
19	`onclause -> ε`	47	`expr -> expr MINUS term`
20	`play -> PLAY comp SEMI`	48	`expr -> term`
21	`comp -> comp BAR seq`	49	`term -> term STAR factor`
22	`comp -> seq`	50	`term -> term SLASH factor`
23	`seq -> seq elem`	51	`term -> factor`
24	`seq -> elem`	52	`factor -> NUMBER`
25	`elem -> REPEAT expr OF elem`	53	`factor -> ID`
26	`elem -> TRANSPOSE expr OF elem`	54	`factor -> MINUS factor`
27	`elem -> SLOW expr OF elem`	55	`factor -> LPAREN expr RPAREN`

4.2 Architecture of the compiler

The compiler is a package of twenty modules, one per concern, driven by a twelve-stage pipeline that keeps every artefact for the user interfaces. The mapping to the syllabus is direct.

Table 4. *Modules of the compiler against the modules of the syllabus*

Module	File	Role
1	`regex_engine.py`	regular expressions to DFA by the direct method; minimisation
1	`tokens.py`	the token specification and its attribute actions
1	`lexer.py`	maximal munch, keyword folding, panic-mode recovery
2	`grammar.py`	productions, FIRST/FOLLOW, LL(1) table, the ambiguous first draft
2	`lr.py`	LR(0)/LR(1) items, SLR, CLR and LALR tables, table-driven parsing
2	`parser_rd.py`	the independent recursive-descent parser
2	`ast_nodes.py`	the abstract syntax tree
3	`sdt.py`	one semantic action per production (S-attributed, run by the LR parser)
3	`semantics.py`	the L-attributed walk, scoped symbol table, every check
4	`ir.py`	quadruples, basic blocks, control-flow graph
4	`irgen.py`	code generation with backpatching and activation-record layouts
5	`optimizer.py`	thirteen passes, each measured
5	`dataflow.py`	live variables, reaching definitions, available expressions, dominators, loops
5	`verify.py`	the abstract-interpretation verifier
6	`vm.py`	the sandboxed virtual machine with activation records
6	`channels.py`	register allocation by graph colouring, registers being MIDI channels
back end	`midi.py`	the Standard MIDI File, byte by byte
driver	`driver.py`	the pipeline and all the metrics

Module	File	Role
tools	`cli.py`, `gui.py`, `report.py`	command line, Tkinter window, HTML report

4.3 Design decisions

- **Direct method, not Thompson plus subset construction.** The combined expression of all 25 rules is one syntax tree; nullable, firstpos, lastpos and followpos give the DFA states directly as sets of positions, and characters that behave identically are grouped into equivalence classes before any transition is computed.
- **One grammar, four parsers.** The SLR, CLR and LALR tables are built from the same item collections so that their sizes can be compared honestly; the LL(1) table is built and allowed to fail, because the grammar is left-recursive by design.
- **Differential parsing.** A hand-written recursive-descent parser produces a second tree for every program; the two dumps must be byte-identical. Any disagreement is reported as a compiler warning.
- **A deliberately naive code generator.** Every note emits the same six-instruction pattern even when the transposition is zero and the stretch is one. The front end emits correct code; the optimiser makes it small. This is what gives Module 5 measurable work.
- **Dynamic scoping through the activation record.** A phrase cannot know its transposition or stretch until it is called, so three implicit parameters (`$off`, `$sn`, `$sd`) are pushed with every call and live in the callee's frame.
- **Channels as registers.** Voices are live ranges, overlap is interference, channel 10 is a pre-coloured node, and seventeen simultaneous voices force a spill.
- **Zero dependencies in the compiler, a strict boundary for the Studio.** Only the `studio` package may import FastAPI, uvicorn and the Groq SDK.

4.4 Cadence Studio

The Studio is a FastAPI application that wraps `compile_source` and serves a single-page interface written in vanilla ES modules with no build step. Its serializer turns every field of the compiler's result into JSON, including the paired note list that the piano roll and the audio scheduler consume and the MIDI bytes as base64. The browser holds a score editor whose highlighter uses the very same maximal-munch policy as the compiler's scanner, an 88-key piano driven by pointer, touch or the computer keyboard, a Web Audio piano synthesiser with an optional sampled grand, a canvas piano roll with a gliding playhead, a capture strip that writes what is played as Cadence tokens, a composer panel, a Glass Box that renders every phase, and a code browser for the project's own source.

Table 5. *The Studio's API*

Endpoint	Purpose
`POST /api/compile`	compile a program; every phase as JSON, MIDI as base64
`POST /api/midi`	compile and download the `.mid` file
`GET /api/examples`, `/api/presets`	the example programs; the ten mood presets with compiled summaries
`GET /api/toolchain`	the DFA and the parse tables, program-independent
`GET /api/core`, `/api/core/{group}/{file}`	the module catalogue and any source file
`GET /api/ai/models`, `POST /api/ai/compose`	the Groq composer; proposals are compiled before they are returned

4.5 Methodology and tools

The compiler was developed phase by phase against the textbook algorithms, each phase landing with tests before the next was started. Correctness arguments that are usually informal were turned into executable checks: the two parsers must agree, every pass must preserve the event stream, the verifier must reject hand-crafted illegal code, and the MIDI output must be readable by an independent reader written for the tests. Tools: Python 3.10 or newer (standard library only for the compiler, `unittest` for the tests); FastAPI, uvicorn, pydantic and the Groq SDK for the Studio server; the Web Audio API, Canvas and ES modules in the browser; Git and GitHub.

5. Implementation & Screenshots:

This section follows the pipeline. Each phase gives the algorithm as implemented, an excerpt of the real code, the measurements the compiler reports for the showcase program, and screenshots of the running software: the Studio's Glass Box, the self-contained HTML report, the Tkinter window and the command line.



Figure 3. *Cadence Studio after compiling the Nocturne preset: score, piano roll, composer and the 88-key piano. The strip under the top bar lights one dot per compiler stage.*

5.1 Module 1: lexical analysis

The token specification is the list of extended regular expressions in Table 2, in priority order. `regex_engine.py` parses each pattern with a small recursive-descent parser into a syntax tree whose leaves are numbered positions, appends an end-marker per rule, and joins the rules into one tree. The bottom-up attributes nullable, firstpos and lastpos and the top-down relation followpos are computed exactly as in the textbook; the DFA's states are sets of positions, and an accepting state is labelled with the rule of its lowest-numbered end marker, which is how rule priority breaks ties. Characters that are accepted by the same set of positions are grouped first, so transitions are computed once per class rather than once per character. The result is minimised by partition refinement.

Listing 2. regex_engine.py: followpos, the top-down half of the direct method

```
def compute_followpos(root, followpos):
    """Two textbook rules only:
         Cat(c1,c2): for every i in lastpos(c1):  followpos(i) |= firstpos(c2)
         Star/Plus(c): for every i in lastpos(c): followpos(i) |= firstpos(c)
    """
    if isinstance(root, Cat):
        for i in root.left.lastpos:
            followpos[i] |= root.right.firstpos
        compute_followpos(root.left, followpos)
        compute_followpos(root.right, followpos)
    elif isinstance(root, Alt):
        compute_followpos(root.left, followpos)
        compute_followpos(root.right, followpos)
    elif isinstance(root, (Star, Plus)):
        for i in root.child.lastpos:
            followpos[i] |= root.child.firstpos
        compute_followpos(root.child, followpos)
    elif isinstance(root, Opt):
        compute_followpos(root.child, followpos)
```

Listing 3. regex_engine.py: DFA states as sets of positions

```
# ---- subset construction over sets of positions ---------------------------
start_set = combined.firstpos
dfa_states = [start_set]
index_of = {start_set: 0}
raw_trans: list[dict] = []
work = [start_set]
while work:
    S = work.pop()
    row = {}
    for positions, chars in groups:
        move = positions & S
        if not move:
            continue
        U = frozenset().union(*(followpos[p] for p in move))
```

```
        if not U:
            continue
        if U not in index_of:
            index_of[U] = len(dfa_states)
            dfa_states.append(U)
            work.append(U)
    ...
```

The scanner runs the DFA with maximal munch: it remembers the last accepting state it passed and, when no transition exists, emits the longest match. `r` alone is a rest because the REST rule is listed before ID; `riff` is an identifier because four characters beat one. Reserved words are recognised by looking an identifier up in a keyword table, which keeps the automaton small. A character no rule can start is reported with line, column and a caret, discarded, and scanning continues, so every lexical error in a file is found in one run.

Table 6. *Lexical analysis as measured by the compiler*

Measurement	Value
token rules	25
numbered positions in the combined tree	101
character equivalence classes	37
DFA states by the direct method	47
DFA states after minimisation	47
reserved words kept out of the DFA	19
tokens scanned in 07_showcase.cad	168

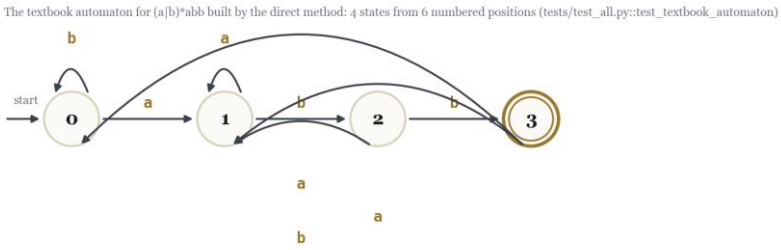


Figure 4. *The direct method on the textbook expression (a|b)*abb gives exactly four states; the test suite checks this.*

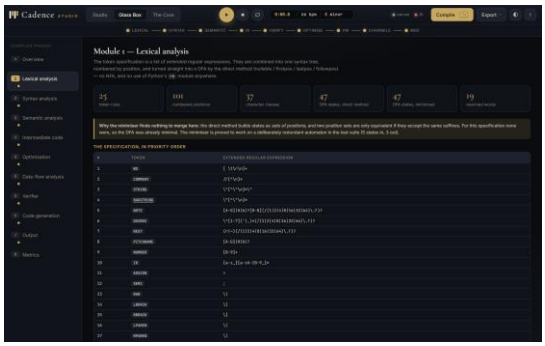


Figure 5. *Glass Box, Module 1: the rules, the DFA statistics and the note on maximal munch versus rule priority.*

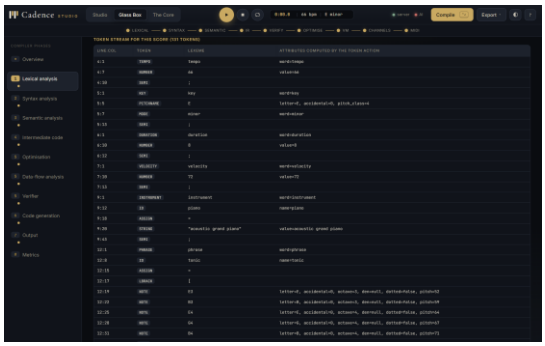


Figure 6. *The token stream of the score with the attributes each token action computed (pitch numbers, denominators, dots).*

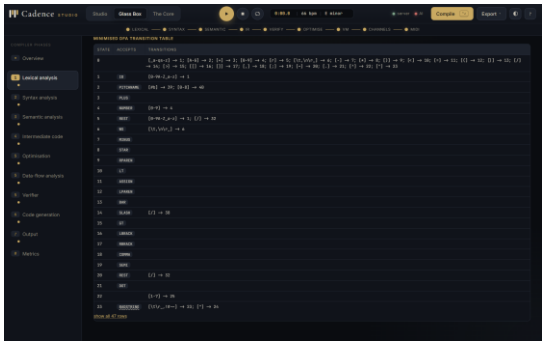


Figure 7. *The minimised transition table, one row per DFA state, with the character classes on each edge.*

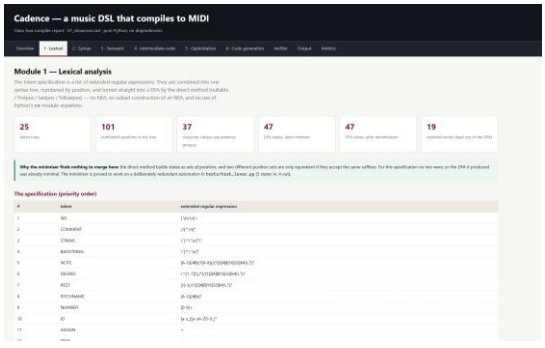


Figure 8. *The same phase in the self-contained HTML report written by report.py.*

5.2 Module 2: syntax analysis

`grammar.py` holds the 56 productions, computes FIRST and FOLLOW by iteration to a fixed point, detects immediate and indirect left recursion by transitive closure, and builds the LL(1) table, which has 56 conflicting entries because eight nonterminals are left-recursive. `lr.py` builds the canonical LR(0) and LR(1) collections from the augmented grammar, then the SLR(1) table using FOLLOW sets as lookahead, the canonical LR(1) table with lookaheads carried in the items, and the LALR(1) table by merging LR(1) states with equal cores. Every shift/reduce or reduce/reduce clash is recorded with the items, the lookahead and the resolution; arithmetic conflicts are resolved by precedence and associativity, and the final grammar has none.

Listing 4. lr.py: LALR(1) is the canonical LR(1) collection with states of equal core merged

```
def lalr_table(self) -> Table:
    """LALR = canonical LR(1) with states of equal CORE merged."""
    states, goto = self.lr1_collection()
    cores = []
    core_index = {}
    for S in states:
        core = frozenset((p, d) for (p, d, _) in S)
        if core not in core_index:
            core_index[core] = len(cores)
            cores.append(core)
    merged = [set() for _ in cores]
    old_to_new = []
    for S in states:
        core = frozenset((p, d) for (p, d, _) in S)
        k = core_index[core]
        merged[k] |= set(S)
        old_to_new.append(k)
    merged = [frozenset(m) for m in merged]
    new_goto = {}
    for (i, sym), j in goto.items():
    ...
```

Table 7. *Parse-table comparison on the project's own grammar*

Parser	States	Table entries	Shift/reduce	Reduce/reduce
SLR(1)	114	742	0	0
CLR(1)	204	1343	0	0
LALR(1)	114	742	0	0
ambiguous first draft (SLR)	28	142	24	2
LL(1) on the final grammar	—	88	56 conflicts	—

CLR(1) needs 204 states because it keeps a separate state for every lookahead context; merging by core gives LALR(1) with 114 states, the same as SLR(1), without reintroducing a single conflict. That is exactly the trade-off LALR exists to make, measured rather than quoted. The first draft of the grammar wrote sequencing as `comp -> comp comp` and used the same brackets for chords and groups; fed to the same table builder it produces 24 shift/reduce and 2 reduce/reduce conflicts. The fix was to change the language, not to hack the table: chords became `< ... >/n` and sequencing got its own left-recursive `seq` nonterminal.

The table-driven parser records a trace of the stack, the remaining input and the action taken; parsing 03_riff.cad takes 93 shifts and 109 reductions. On an error it reports the tokens that would have been legal in that state, read straight out of the table, then recovers in panic mode to the next semicolon and pops to a state that can shift a declaration, so later errors are still found. Semantic actions in `sdt.py`, one per production and checked against the grammar at import time, build the tree bottom-up. The recursive-descent parser in `parser_rd.py` builds the same tree with left recursion turned into iteration; the test suite compares the two dumps for every example.

Listing 5. lr.py: statement-level panic-mode recovery

```
def _recover(table, state_stack, value_stack, toks, i):
    """
    Panic mode, statement level:  throw away input up to and including the next ';'
    then pop the stack down to a state that can shift a fresh declaration.
    """
    n = len(toks)
    while i < n and toks[i].kind not in ("SEMI", "$"):
        i += 1
    if i < n and toks[i].kind == "SEMI":
        i += 1
    if i >= n or toks[i].kind == "$":
        return i, False
    while len(state_stack) > 1:
        if any((state_stack[-1], sym) in table.goto for sym in ("decls", "decl")):
            break
```

```
        state_stack.pop()
```

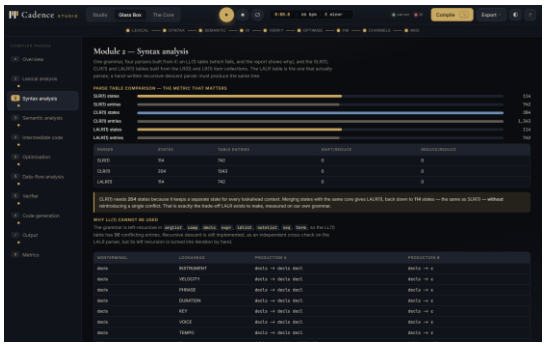


Figure 9. *Glass Box, Module 2: the parse-table comparison drawn as bars, and why LL(1) cannot be used.*

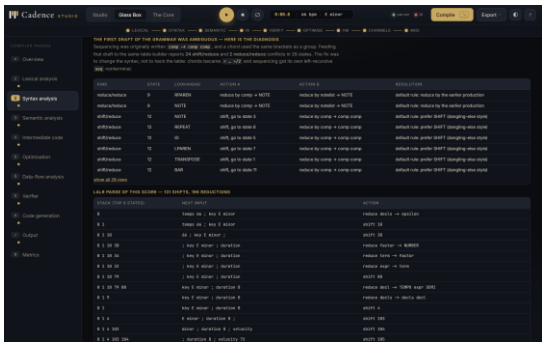


Figure 10. *The diagnosis of the ambiguous first draft, with each conflict's state, lookahead and resolution.*

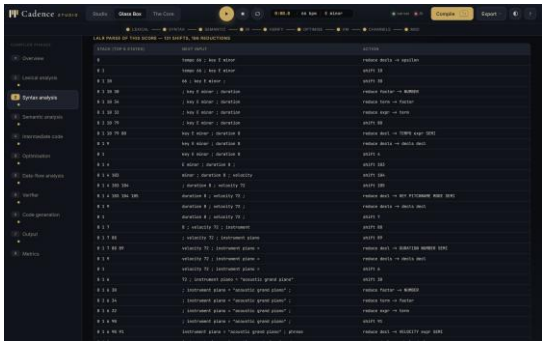


Figure 11. *The LALR parse of the score: stack, next input and action, step by step.*

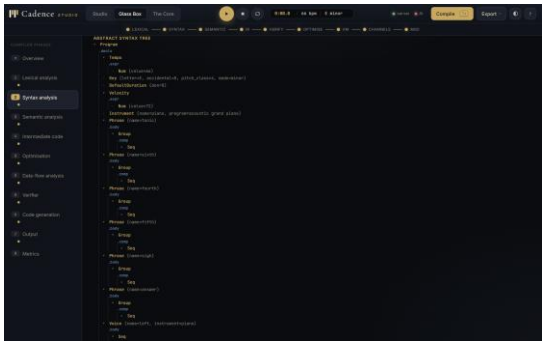


Figure 12. *The abstract syntax tree as an interactive tree; every node carries its line and column.*

Listing 6. The compiler's diagnostics for examples/bad_syntax.cad (a missing semicolon and a missing bracket)

```
[syntax error] line 8:1: unexpected 'key'
    key C major ;
    ^
    hint: expected ',', '-', 'of', '+', ')', ';' ...
[syntax error] line 10:34: unexpected ';'
    voice tune on piano = [ C4 E4 G4 ;
                                     ^
    hint: expected '|', ']'
[semantic error] line 11:6: 'tune' is not declared
    play tune ;
         ^


3 error(s) -- no MIDI file written
```

5.3 Module 3: semantic analysis

The analyser is an L-attributed syntax-directed definition. An immutable context record carries the inherited attributes down the tree: key and mode, octave, default duration, velocity, the accumulated transposition and the accumulated stretch. Every child receives a fresh copy with its own changes applied. A synthesised record carries ticks, note count and pitch range back up. The analyser makes two passes over the declarations (names first, then settings), builds the phrase call graph syntactically before the walk so that recursion is diagnosed instead of looping, and then checks every phrase and voice.

Listing 7. semantics.py: the inherited-attribute record

```
@dataclass(frozen=True)
class Ctx:
    """The INHERITED attribute record.  Immutable: every child gets a fresh copy."""
    key_pc: int = 0
    mode: str = "major"
    octave: int = 4
    den: int = 4
    dotted: bool = False
    velocity: int = 90
    transpose: int = 0          # None means "not known until run time"
```

```
    stretch_n: int = 1
    stretch_d: int = 1
    inside: str = "global"      # 'phrase <name>' or 'voice <name>'
    percussion: bool = False


    def brief(self):
        t = "?" if self.transpose is None else self.transpose
        return (f"key={PC_NAMES[self.key_pc]} {self.mode}, oct={self.octave}, "
                f"dur=1/{self.den}{'.' if self.dotted else ''}, vel={self.velocity}, "
                f"transpose={t}, stretch={self.stretch_n}/{self.stretch_d}")
```

Listing 8. semantics.py: transposition accumulates on the way down; a degree becomes a pitch only when the key reaches it

```
def _Transpose(self, n, ctx, scope, depth):
    amount = self.fold(n.amount, scope)
    # INHERITED attribute: the accumulated transposition passed down
    newt = None if (amount is None or ctx.transpose is None) else ctx.transpose + amount
    s = self.check(n.body, replace(ctx, transpose=newt), scope, depth + 1)
    return self.note(depth, n, ctx, s)


def _DegreeAtom(self, n, ctx, scope, depth):
    if not 1 <= n.degree <= 7:
        self.err(n, f"scale degree {n.degree} does not exist")
        return Syn(0, 0)
    interval = SCALES[ctx.mode][n.degree - 1]
    pitch = 12 * (ctx.octave + 1 + n.octave_shift) + ctx.key_pc + interval
    p = self._pitch_out(n, ctx, pitch)
    return self.note(depth, n, ctx, Syn(self._len(n, ctx), 1, p, p))
```

Checks performed: undeclared and duplicate names, wrong argument counts, recursive and mutually recursive phrases, voices used inside phrases, parameters played as music, tempo, velocity, duration and program ranges, pitches pushed outside 0 to 127 by transposition, unknown instrument names (with suggestions by edit distance over the complete General MIDI list), division by zero in constant expressions, note lengths inside chords, unused phrases and unplayed voices. Every diagnostic carries line, column, a caret and a hint.

Table 8. *Semantic analysis of the showcase program*

Setting or count	Value for 07_showcase.cad
key	E minor
tempo	100
default duration	1/8
velocity	92
phrases / voices / instruments	4 / 4 / 4
symbols in all scopes	14
warnings	0

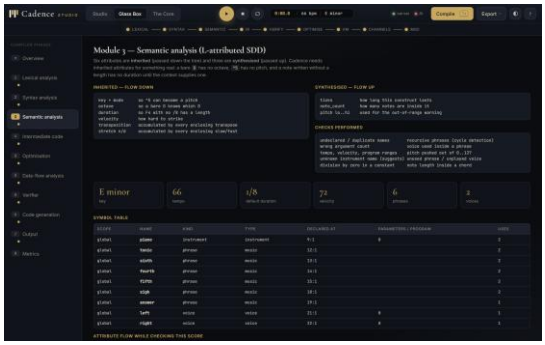


Figure 13. *Glass Box, Module 3: the inherited and synthesised attributes and the checks performed.*

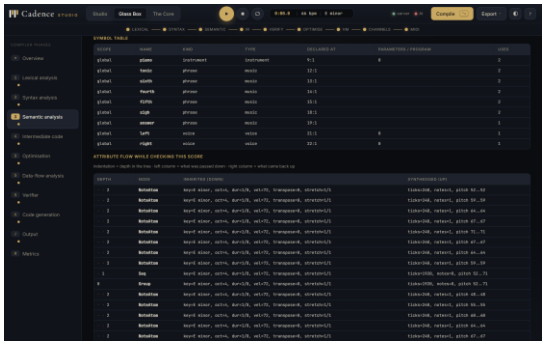


Figure 14. *The scoped symbol table with kinds, types, declaration positions and use counts.*

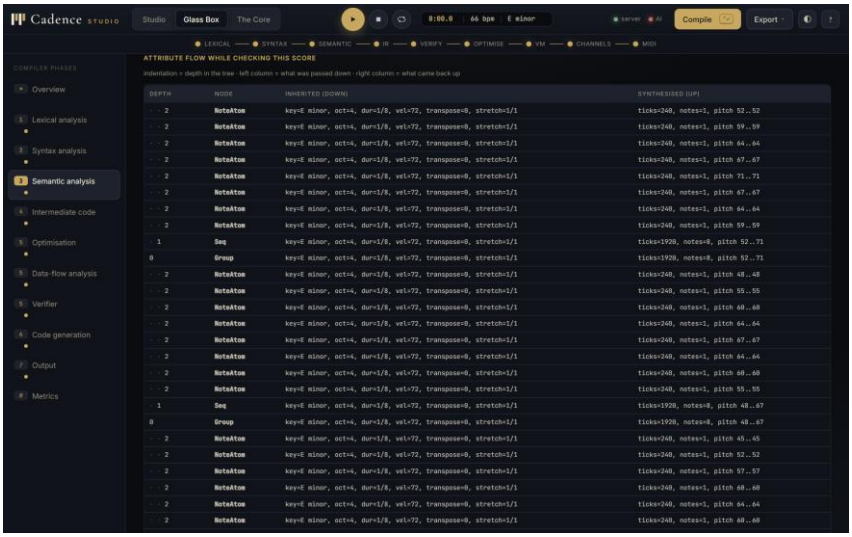


Figure 15. *The attribute flow while checking the score: what was passed down and what came back up at every node.*

Listing 9. Diagnostics for examples/bad_semantic.cad: every line is legal syntax; only the analyser can reject it

```
[semantic error] line 12:1: tempo 900 is outside the playable range 20-400
    tempo 900 ;
    ^
[semantic error] line 14:1: unknown instrument name 'grand pianoo'
    instrument piano = "grand pianoo" ;
    ^
    hint: did you mean: 'grand piano', 'bright piano', 'piano'?
[semantic error] line 17:1: phrase 'loop' is recursive (loop -> loop)
    phrase loop    = [ C4 loop ] ;
    ^
    hint: a phrase cannot play itself: the music would never end
[semantic warning] line 17:1: phrase 'loop' is declared but never used
    phrase loop    = [ C4 loop ] ;
    ^
    hint: dead code elimination will drop it
[semantic error] line 19:1: 'plano' is not a declared instrument
    voice tune on plano = riff(1, 2) ;
    ^
    hint: add:  instrument plano = 0 ;
[semantic error] line 19:23: phrase 'riff' takes 1 argument(s) but got 2
    voice tune on plano = riff(1, 2) ;
                          ^
    hint: it is declared on line 16 as riff(t)
[semantic warning] line 20:41: pitch 198 is outside MIDI range 0-127
    ...
```

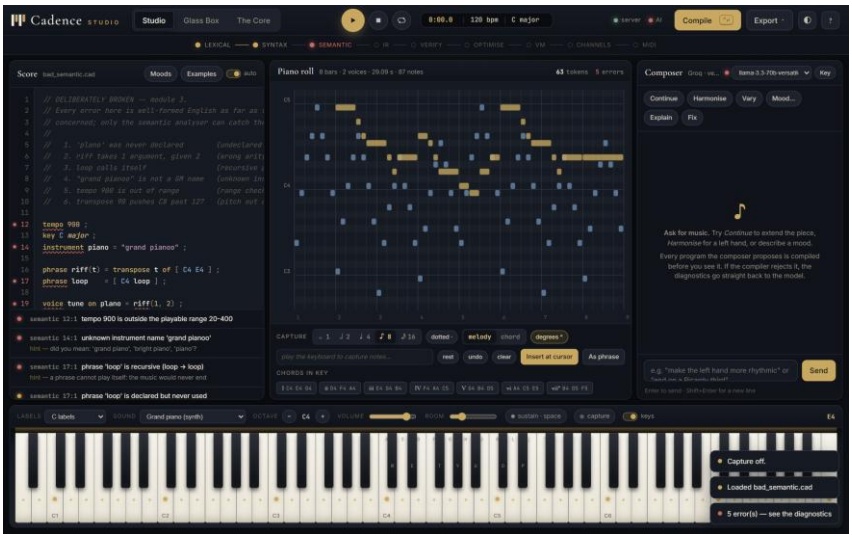


Figure 16. *The same broken program in the Studio: errors under the editor and on their line numbers, the pipeline strip stopping at the semantic stage.*

5.4 Module 4: intermediate code generation

`irgen.py` walks the annotated tree and emits three-address code in quadruple form. Two instruction families live side by side: ordinary compiler quads (`t1 = 62 + $off`, `if t2 <= 0 goto L3`) and musical quads (`NOTEON t1, 90`, `DELAY 240`, `NOTEOFF t1`, `PARBEGIN`, `PARNEXT`, `PAREND`). Each phrase becomes a procedure whose parameters are the three implicit context words plus its own, and whose activation-record layout (return address, control link, parameters, locals) is computed from the temporaries it defines. `repeat n of X` is a loop whose exit jump is emitted before its target label exists: the jump is remembered in a list and patched when the label is generated, which is textbook backpatching.

Listing 10. irgen.py: a repeat loop closed by backpatching

```
def _Repeat(self, n, ctx, params):
    count = self.expr(n.count, ctx, params)
    counter = self.new_temp()
    self.emit("COPY", count, res=counter, note="loop counter")
    top = self.new_label()
    self.emit("LABEL", top)
    hole = self.makelist(self.emit("IFLE", counter, 0, None,
                                   "exit when the counter runs out"))
    self.gen(n.body, ctx, params)
    self.emit("SUB", counter, 1, counter, "decrement")
    self.emit("GOTO", top)
    after = self.new_label()
    self.emit("LABEL", after)
    self.backpatch(hole, after)          # <-- the backpatch
    return
```

Listing 11. irgen.py: the deliberately naive six-instruction pattern for every note

```
def sound(self, pitches, den, dotted, ctx, label):
    ts = []
    for p in pitches:
        t = self.arith("ADD", p, ctx.off, f"{label}: apply transposition")
        self.emit("NOTEON", t, ctx.velocity, note=label)
        ts.append(t)
    d = self.duration(den, dotted, ctx, label)
    self.emit("DELAY", d, note=label)
    for t in reversed(ts):
        self.emit("NOTEOFF", t, note=label)
```

Listing 12. Naive three-address code for 03_riff.cad (python -m cadence examples/03_riff.cad --dump ir)

```
--- phrase riff($off, $sn, $sd, $t) ----------------------------
   0      t1 = $off + $t   ; accumulate transposition
   1      t2 = 62 + t1   ; D4: apply transposition
   2      NOTEON  t2, vel 96   ; D4
   3      t3 = 240 * $sn   ; D4: scale by stretch
   4      t4 = t3 / $sd   ; D4: scale by stretch
   5      DELAY   t4   ; D4
   6      NOTEOFF t2   ; D4
   7      t5 = 65 + t1   ; F4: apply transposition
   8      NOTEON  t5, vel 96   ; F4
   9      t6 = 240 * $sn   ; F4: scale by stretch
  10      t7 = t6 / $sd   ; F4: scale by stretch
  11      DELAY   t7   ; F4
  12      NOTEOFF t5   ; F4
  13      t8 = 69 + t1   ; A4: apply transposition
  14      NOTEON  t8, vel 96   ; A4
  15      t9 = 240 * $sn   ; A4: scale by stretch
  16      t10 = t9 / $sd   ; A4: scale by stretch
  17      DELAY   t10   ; A4
  18      NOTEOFF t8   ; A4
  19      t11 = 62 + t1   ; chord of 3: apply transposition
  20      NOTEON  t11, vel 96   ; chord of 3
  21      t12 = 65 + t1   ; chord of 3: apply transposition
  22      NOTEON  t12, vel 96   ; chord of 3
    ...
```

Table 9. *Intermediate code of the showcase program before and after optimisation*

Measurement	Value
procedures (naive)	9
static instructions (naive)	173
procedures (optimised)	5
static instructions (optimised)	247
static change	+42.8%

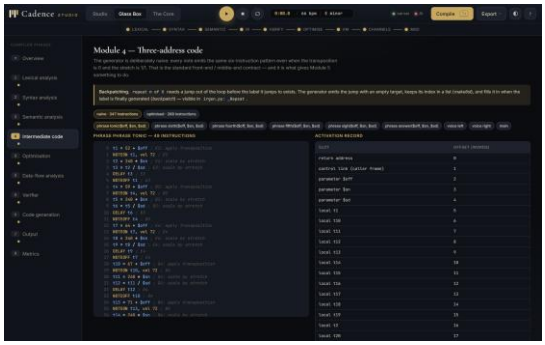


Figure 17. *Glass Box, Module 4: the listing of one procedure with its activation-record layout.*



Figure 18. *The HTML report's Module 4 page: the naive code and the frame of the first phrase.*

5.5 Module 5: optimisation

The optimiser runs the passes below in order at -O2, each measured twice: statically, by counting the instructions that exist, and dynamically, by re-running the virtual machine and counting the instructions it executes. Inlining and unrolling make the code larger and the work smaller; that trade-off is the point. After each pass the event stream is compared with the unoptimised run. Two further passes that change articulation (tying repeated notes and dropping zero-length notes) are off by default and excluded from the equivalence check.

Listing 13. optimizer.py: the pass pipeline, and the measurement hook that re-runs the VM after each pass

```
def run(self):
    if self.level <= 0:
        return self.results
    pipeline = [self.constant_folding, self.copy_propagation, self.peephole]
    if self.level >= 2:
        pipeline = [
            self.cse_dag,                  # before folding, while the DAG has work
            self.inline_phrases,
            self.cse_dag,
            self.constant_folding, self.copy_propagation,
            self.loop_unrolling,
            self.cse_dag,
            self.constant_folding, self.copy_propagation,
            self.strength_reduction,
            self.dead_code_elimination,
            self.remove_unused_procedures,
            self.peephole,
            self.dead_code_elimination,
        ]
    if self.articulate:
        pipeline += [self.tie_merge, self.drop_zero_length]
    for p in pipeline:
        before = self.prog.real_count()
        changes, detail = p()
    ...
```

Listing 14. driver.py: the per-pass proof that the music did not change

```
baseline_sig = event_signature(r.events0)


def measure(pass_name, prog):
    probe = VM(copy.deepcopy(prog), [])
    ev = probe.run_all()
    return {
        "steps": probe.steps,
        "same_music": (event_signature(ev) == baseline_sig) if not articulate else None,
    }
```

Table 10. *Pass-by-pass measurement on 07_showcase.cad, as reported by the compiler*

Pass	Static before	After	Δ	Rewrites	Executed	Music same	Detail
cse dag	173	173	+0	15	631	yes	15 repeated expression(s) reused from the DAG
inline phrases	173	276	-103	5	542	yes	5 call site(s) inlined
cse dag	276	276	+0	1	542	yes	1 repeated expression(s) reused from the DAG
constant folding	276	276	+0	45	542	yes	45 expression(s) folded or simplified
copy propagation	276	276	+0	124	542	yes	124 operand(s) replaced by their source
loop unrolling	276	567	-291	4	462	yes	4 loop(s) unrolled
cse dag	567	567	+0	20	462	yes	20 repeated expression(s) reused from the DAG
constant folding	567	567	+0	8	462	yes	8 expression(s) folded or simplified
copy propagation	567	567	+0	71	462	yes	71 operand(s) replaced by their source
strength reduction	567	567	+0	0	462	yes	0 multiplication(s) turned into addition
dead code elimination	567	325	+242	242	241	yes	242 dead instruction(s) removed
remove unused procedures	325	247	+78	4	241	yes	removed phrase motif, phrase answer, phrase pedal, phrase beat
peephole	247	247	+0	0	241	yes	0 peephole rewrite(s)

Table 11. *What the optimiser achieved on the showcase program*

Measurement	Value
instructions executed, naive	631
instructions executed, optimised	241
work saved	61.8%
optimiser preserved the music	True
deepest activation-record stack	2

Common subexpressions are found by value-numbering each basic block into a DAG; the excerpt of the DAG the compiler prints for the showcase program is shown in the Glass Box below. Loop unrolling recognises the exact shape the generator emits for `repeat`, requires a constant trip count and a body that does not touch the counter, and renames labels per copy. Dead-code elimination is driven by per-instruction liveness; instructions with an observable musical effect are never removed. The peephole pass merges adjacent delays, removes `DELAY 0`, jumps to the next instruction and jump-to-jump chains.

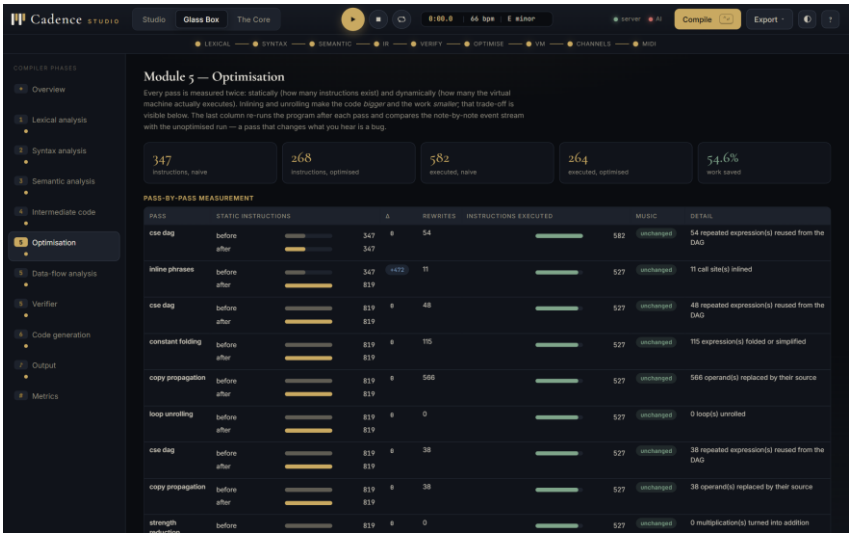


Figure 19. *Glass Box, Module 5: the pass table with static and dynamic bars, and the music-unchanged column.*

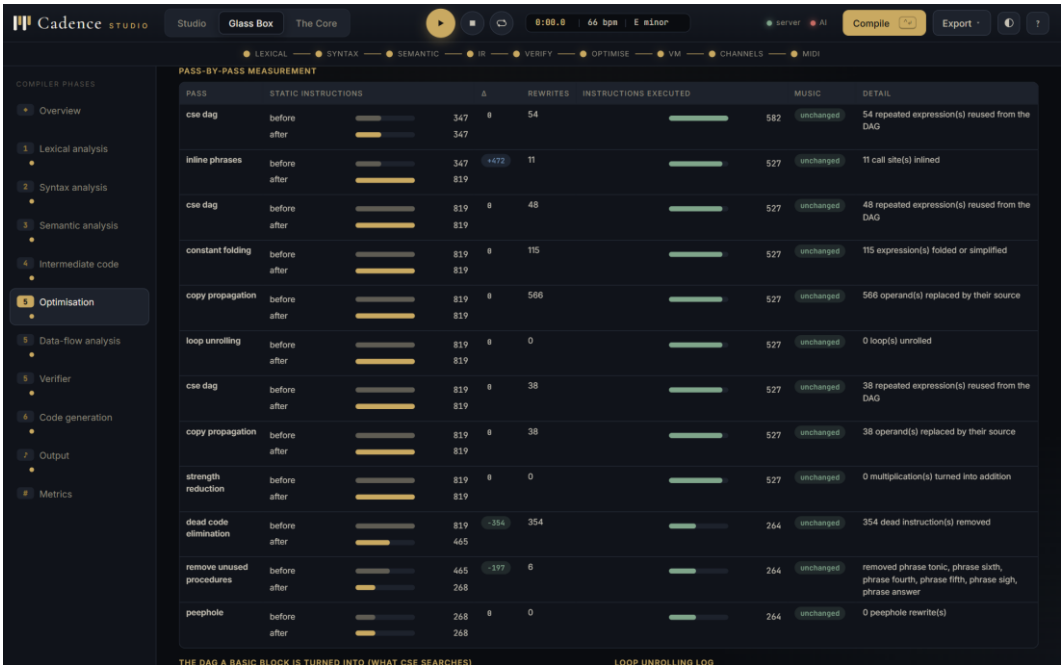


Figure 20. *The same table scrolled to the later passes; every pass reports its rewrites and its re-run.*

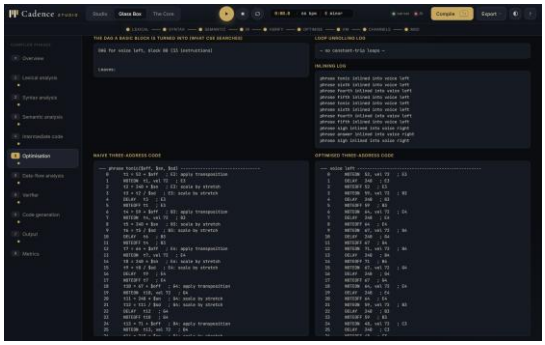


Figure 21. *The DAG of a basic block, the structure common-subexpression elimination searches, with the unrolling and inlining logs.*

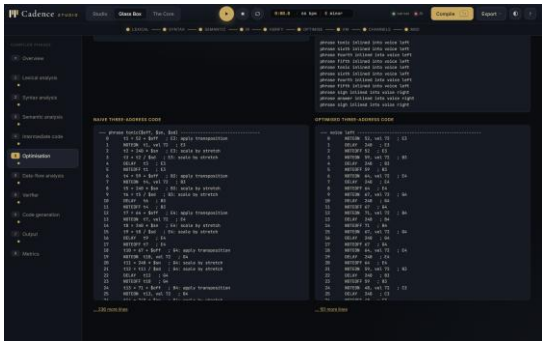


Figure 22. *Naive and optimised three-address code side by side.*

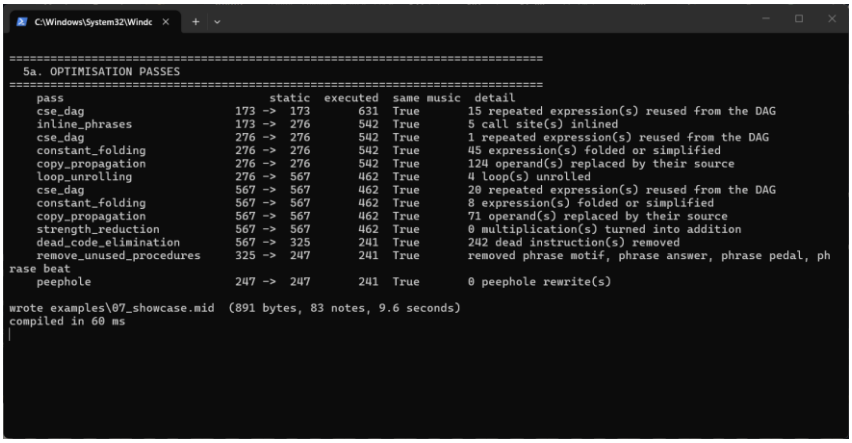


Figure 23. *The command line: python -m cadence examples/07_showcase.cad --dump passes.*

5.6 Module 5: data-flow analysis

`dataflow.py` implements the classical iterative analyses over basic blocks, keeping the IN and OUT sets of every round so that the fixed point can be shown being reached: live variables (backward, union), which drives dead-code elimination; reaching definitions (forward, union); available expressions (forward, intersection), which drives common-subexpression elimination; and dominators (forward, intersection), from which back edges and

natural loops are found. The driver runs them on the most interesting procedure of each program, preferring a voice with a loop.

Listing 15. dataflow.py: live variables, round by round

```
def live_variables(proc: Proc, blocks: list[Block]):
    use, defs = {}, {}
    for b in blocks:
        u, d = set(), set()
        for q in b.quads:
            for x in q.uses():
                if x not in d:
                    u.add(x)
            d.update(q.defines())
        use[b.index], defs[b.index] = u, d
    IN = {b.index: set() for b in blocks}
    OUT = {b.index: set() for b in blocks}
    log = []
    for rnd in range(1, 60):
        changed = False
        for b in reversed(blocks):
            newout = set()
            for s in b.succ:
                newout |= IN[s]
            newin = use[b.index] | (newout - defs[b.index])
            if newout != OUT[b.index] or newin != IN[b.index]:
                OUT[b.index], IN[b.index] = newout, newin
    ...
```

Listing 16. The control-flow graph of voice lead in 03_riff.cad with its back edge, dominators and natural loop

```
B0  (starts at 0)
        t1 = 2
    --> B1


B1  (starts at 1, label L1)
        L1:
        if t1 <= 0 goto L2
    --> B3, B2


B2  (starts at 3)
        PARAM   0
        PARAM   1
        PARAM   1
        PARAM   0
        CALL    phrase riff, 4
        t1 = t1 - 1
        goto L1
    --> B1 [back edge]


B3  (starts at 10, label L2)
    ...
```

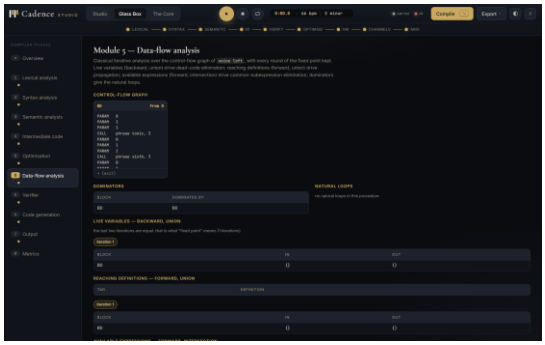


Figure 24. *Glass Box: the control-flow graph as block cards with successors and back edges marked.*

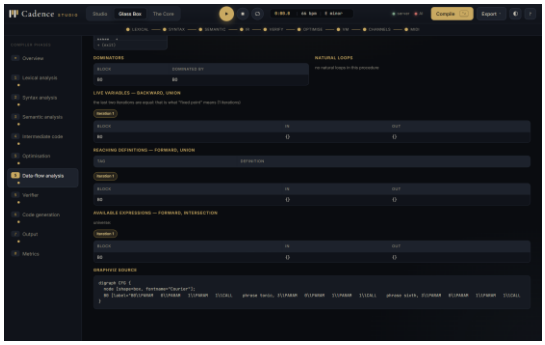


Figure 25. *Dominators and natural loops of the showcase procedure.*

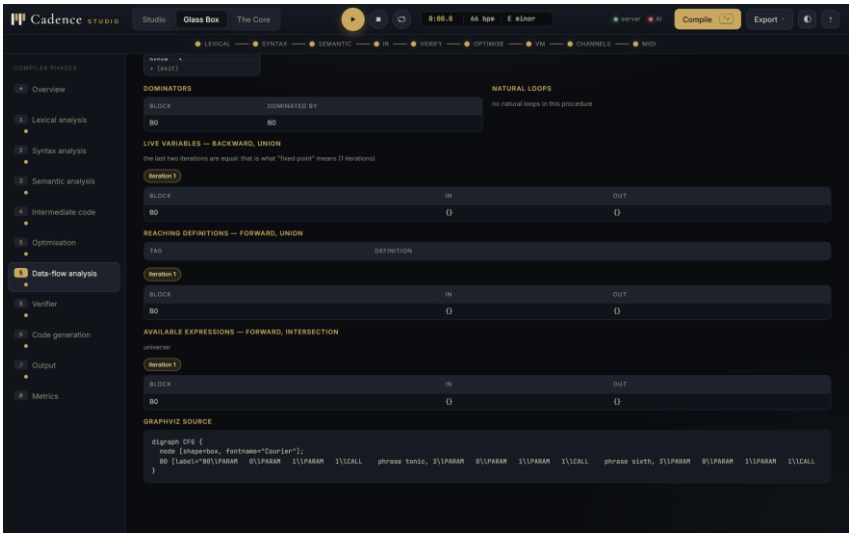


Figure 26. *Live variables: the IN and OUT sets, one tab per iteration until the last two agree.*

5.7 Module 5: security checking of virtual-machine code

Before the VM is allowed to execute an instruction, `verify.py` checks the generated code by abstract interpretation over the control-flow graph: every jump target exists in the same procedure; every CALL names a procedure that exists, with the right number of arguments, and enough PARAMs are pushed on every path reaching it; the PARBEGIN/PAREND nesting depth agrees at every merge point, which is the same check a JVM verifier makes on operand-stack depth; no local is read on any path where it has not been written, a forward definitely-assigned analysis with intersection as the meet; and every procedure ends in RET. The test suite hand-crafts illegal code for each check and confirms it is rejected; the VM's step, frame and event limits are tested with an infinite loop that must be stopped rather than hang.

Listing 17. verify.py: the definitely-assigned analysis

```
# ---- 4. definitely-assigned analysis ----------------------------------
universe = {x for q in p.code for x in q.defines()} | set(p.params)
IN = {b.index: (set(universe) if b.index else set(p.params)) for b in blocks}
changed = True
rounds = 0
while changed and rounds < 200:
    changed, rounds = False, rounds + 1
    for b in blocks:
        if b.index == 0:
            newin = set(p.params)
        else:
            preds = [IN_OUT(IN, blocks, pi) for pi in b.pred]
            newin = set.intersection(*preds) if preds else set(p.params)
        if newin != IN[b.index]:
            IN[b.index] = newin
            changed = True
unassigned = []
for b in blocks:
    live = set(IN[b.index])
    for q in b.quads:
    ...
```

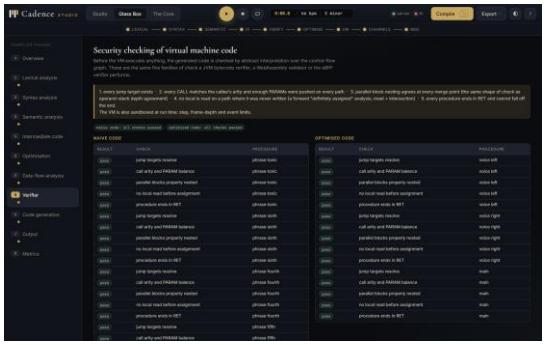


Figure 27. *Glass Box: the five checks, each passed for every procedure of both the naive and the optimised code.*

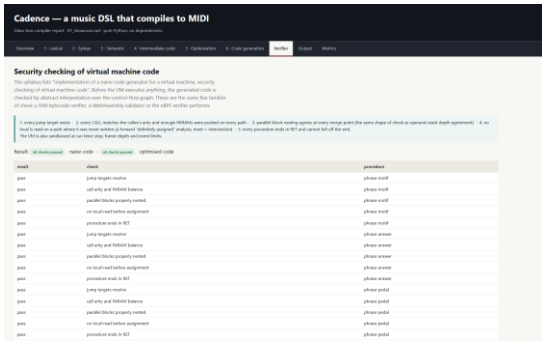


Figure 28. *The verifier page of the HTML report.*

5.8 Module 6: code generation, activation records and register allocation

The target is a small virtual machine. `main` sets the tempo and spawns each played voice; a voice runs with a frame stack on which every phrase call pushes a frame holding the return address, the control link, the parameters and the locals, and pops it on RET. The frame stack is recorded as the program runs, which is what the Glass Box and the report draw. Parallel blocks keep a stack of time cursors: PARBEGIN saves the current time, PARNEXT rewinds to it, and PAREND advances to the end of the longest branch. NOTEON and NOTEOFF produce the timed event stream that is both the thing the optimiser must preserve and the input to the back end.

Listing 18. vm.py: a phrase call pushes an activation record

```
if op == "CALL":
    callee = self.prog.procs.get(q.a1)
    if callee is None:
        self.errors.append(CompileError("runtime", f"no code for {q.a1}"))
        staged.clear()
        continue
    if len(frames) >= self.MAX_FRAMES:
        self.aborted = True
        self.errors.append(CompileError(
            "runtime", "call depth limit reached",
            hint=f"the sandbox allows {self.MAX_FRAMES} frames"))
        break
    args = staged[-q.a2:] if q.a2 else []
    staged = staged[:len(staged) - len(args)]
    nf = Frame(callee.name, pc, len(frames) - 1,
               dict(zip(callee.params, args)))
    frames.append(nf)
    self.max_depth = max(self.max_depth, len(frames))
    if len(self.ar_snapshots) < 40:
        self.ar_snapshots.append(
    ...
```

A MIDI file has sixteen channels and General MIDI reserves channel 10 for percussion, so a program with more simultaneous voices than channels is the register-allocation problem. `channels.py` computes live ranges per voice, builds the interference graph from overlapping half-open ranges, pre-colours percussion voices to channel 10, and runs Chaitin's simplify-and-select. When nothing simplifies, the voice with the fewest notes is chosen as a spill candidate; a real spill makes two voices share a channel, and the report warns that overlapping notes may cut each other off. Example 06_spill.cad puts seventeen voices into fifteen usable channels on purpose.

Listing 19. channels.py: simplify and select

```
# ---- Chaitin: simplify ------------------------------------------------------
degree = {v.name: 0 for v in vs}
adj = {v.name: set() for v in vs}
for a, b in alloc.edges:
    adj[a].add(b)
    adj[b].add(a)
    degree[a] += 1
    degree[b] += 1
remaining = {v.name for v in vs if not v.percussion}
stack = []
work = dict(degree)
while remaining:
    pick = next((n for n in sorted(remaining)
                 if len({m for m in adj[n] if m in remaining}) < k), None)
    if pick is None:
        # nothing simplifies: choose a spill candidate (fewest notes loses)
        pick = min(remaining, key=lambda n: _by_name(vs, n).notes)
        alloc.spills.append(pick)
        alloc.order.append((pick, "potential spill"))
    else:
        alloc.order.append((pick, f"degree {len({m for m in adj[pick] if m in remaining})}"))
    stack.append(pick)
    ...
```

Table 12. *Channel allocation for the showcase program; the drum kit is pre-coloured to channel 10*

Voice	GM program	Live range (ticks)	Notes	Allocated
melody	27	0..7200	23	channel 2
chords	89	0..7680	12	channel 3
under	35	0..7680	8	channel 1
groove	percussion	0..7680	40	channel 10

`midi.py` then writes the Standard MIDI File, format 1: a header chunk, a tempo track carrying the tempo and time-signature meta events, and one track per voice with a program change, note-on and note-off messages and variable-

length delta times, ending in the end-of-track meta event. The test suite parses every produced file back with an independent reader and checks that every note that starts also stops and that the tempo reached the file.

Listing 20. midi.py: variable-length quantities and the per-voice track

```
def vlq(n: int) -> bytes:
    """Variable-length quantity: 7 bits per byte, high bit set on all but the last."""
    if n < 0:
        n = 0
    out = [n & 0x7F]
    n >>= 7
    while n:
        out.append((n & 0x7F) | 0x80)
        n >>= 7
    return bytes(reversed(out))


# ---- one track per voice -------------------------------------------------
for t in tracks:
    data = b"\x00" + meta(0x03, t["name"].encode("ascii", "replace"))
    events = sorted(t["events"], key=lambda e: (e[0], 0 if e[1] != "on" else 1))
    last = 0
    for time, kind, pitch, vel in events:
        delta = max(0, time - last)
        last = time
        ch = t["channel"] & 0x0F
        if kind == "program":
            data += vlq(delta) + bytes([0xC0 | ch, pitch & 0x7F])
    ...
```

Table 13. *The back end on the showcase program*

Measurement	Value
voices	4
channels used	4
channel spills	0
note events	83
length	7680 ticks, 9.6 s
MIDI file size	891 bytes

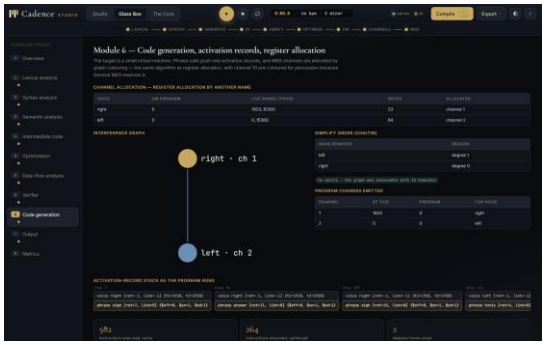


Figure 29. *Glass Box, Module 6: channel allocation, the interference graph drawn as a circle of voices, and the simplify order.*

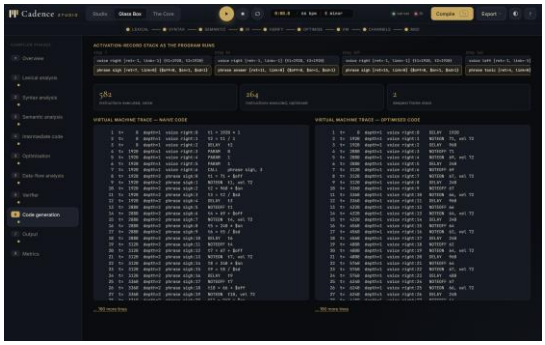


Figure 30. *The activation-record stack as the program runs, one column per snapshot, innermost frame highlighted.*

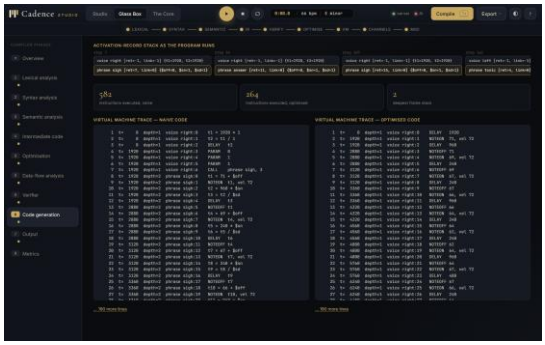


Figure 31. *The virtual-machine traces of the naive and the optimised code.*

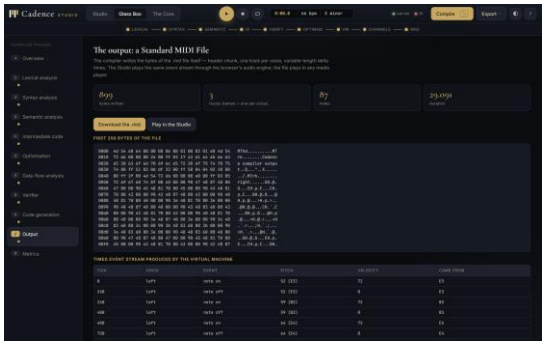


Figure 32. *The output page: the file's size, its tracks and a download; the Studio plays the same event stream.*

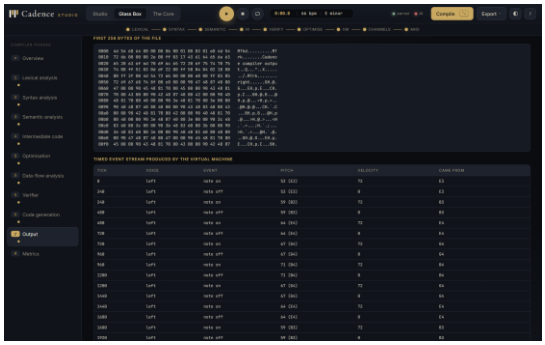


Figure 33. *The first 256 bytes of the .mid file: MThd, the tempo track, MTrk chunks.*

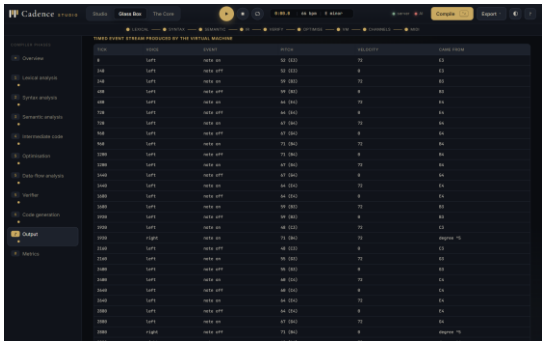


Figure 34. *The timed event stream produced by the VM, with the source note each event came from.*



Figure 35. *The HTML report's piano roll of the compiled showcase program, one colour per voice.*

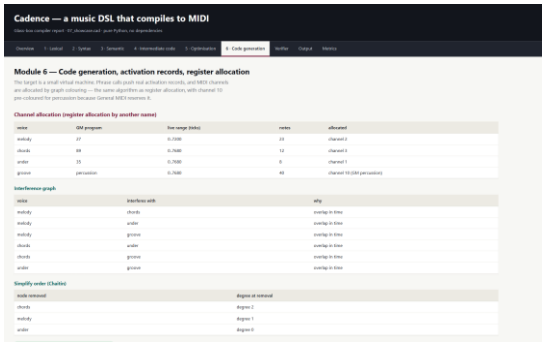


Figure 36. *The HTML report, Module 6: channel allocation, the interference graph and the activation-record stack.*

5.9 The command line, the Tkinter window and the HTML report

Every phase can be printed from the command line with `--dump`, compiled in batch with `--all`, or written as a self-contained HTML report with `--report`. `run.bat` opens the Tkinter Glass-Box window with one tab per phase; `build_all.bat` compiles every example, runs the tests and opens the report.

Listing 21. python -m cadence --all: every example compiles, every deliberately broken one is rejected in the right phase

```
file                    notes     sec  instr naive  executed   saved  status
01_hello.cad                4    3.12           28        13   48.0%  ok
02_scales.cad              16    4.25          110        49   54.2%  ok
03_riff.cad                27    3.75           83        82   59.2%  ok
04_canon.cad               63   11.11           84       194   59.8%  ok
05_drums.cad               34    3.64           82        98   59.2%  ok
06_spill.cad              122    3.43          197       383   71.8%  ok
07_showcase.cad            83     9.6          173       241   61.8%  ok
bad_lexical.cad             -       -            -         -       -  rejected as expected: lexical
bad_semantic.cad            -       -            -         -       -  rejected as expected: semantic
bad_syntax.cad              -       -            -         -       -  rejected as expected: syntax

10 behaved as expected, 0 did not
```

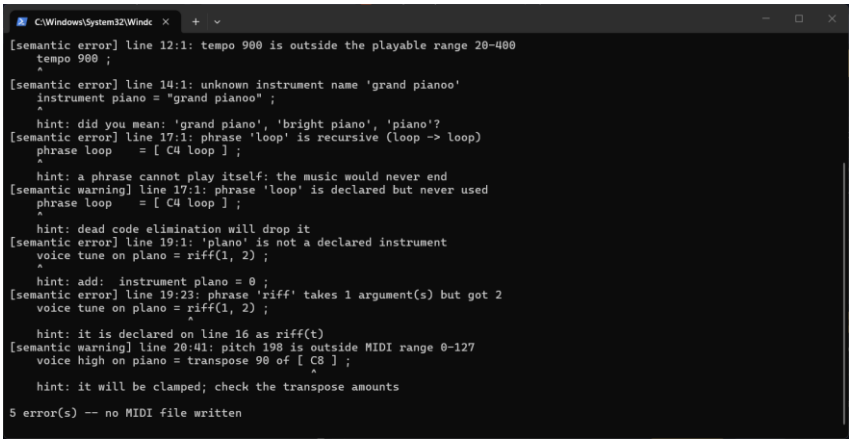


Figure 37. *The command line: batch compilation followed by the diagnostics of the deliberately broken semantic example.*

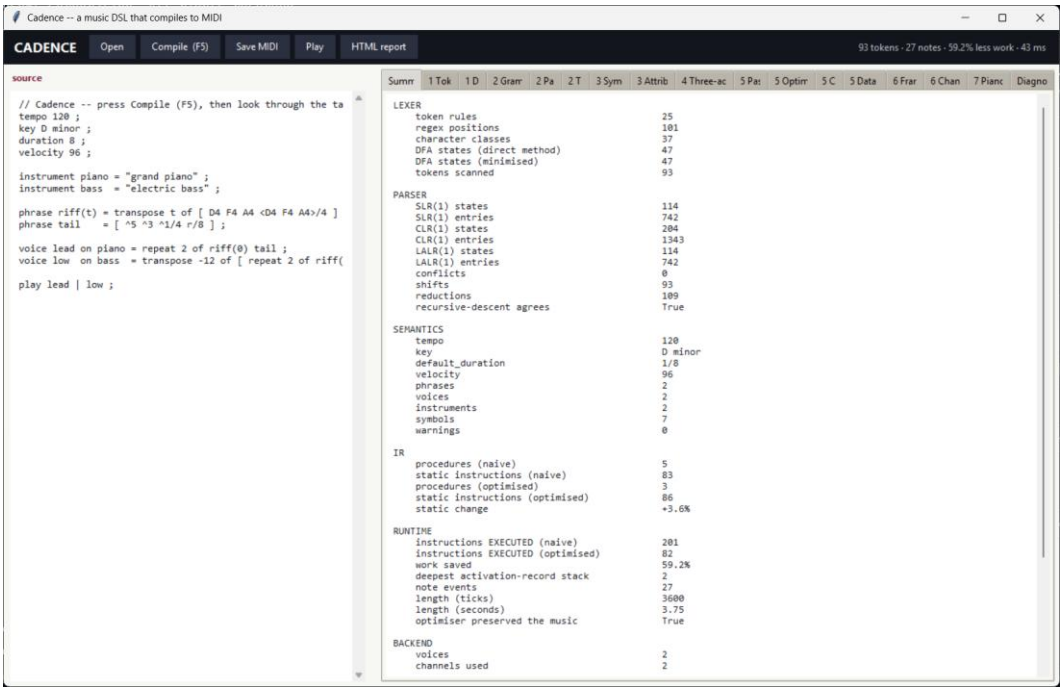


Figure 38. *The Tkinter Glass-Box window (python -m cadence --gui): source on the left, one tab per phase on the right.*

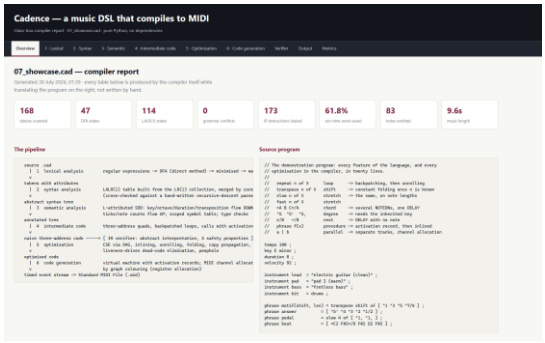


Figure 39. *The self-contained HTML report: overview with the measured cards and the pipeline.*

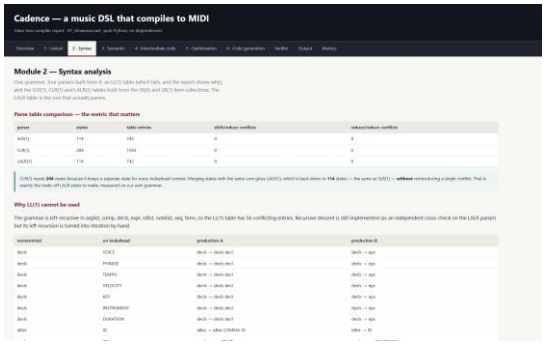


Figure 40. *The HTML report, Module 2: parse-table comparison and the conflict diagnosis.*

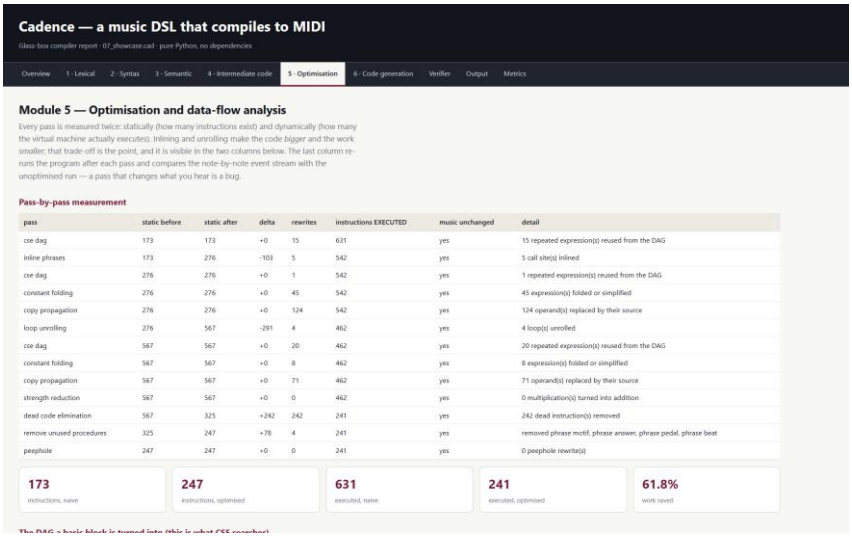


Figure 41. *The HTML report, Module 5: pass measurements and the data-flow fixed points.*

5.10 Cadence Studio

The Studio is served by `studio/server.py` on uvicorn. `POST /api/compile` runs `compile_source` and returns every field of the result as JSON through `serialize.py`, including the stages reached, the paired note list and the MIDI bytes. The page is three views. **Studio** holds the score editor, the piano roll, the composer and the piano; **Glass Box** renders every phase of the current score, which is where the Glass Box screenshots above come from; **The Core** explains the pipeline, shows live measurements and lets the reader open any source file of the compiler or the Studio.

Listing 22. server.py: the compile and MIDI endpoints

```
@app.post("/api/compile")
def compile_endpoint(req: CompileRequest):
    r = compile_source(req.source, req.name or "score.cad", level=req.level,
                       articulate=req.articulate)
    return result_json(r, include_midi=req.midi)



@app.post("/api/midi")
def midi_endpoint(req: CompileRequest):
    r = compile_source(req.source, req.name or "score.cad", level=req.level,
                       articulate=req.articulate, want_dataflow=False)
    if not r.ok():
        raise HTTPException(422, detail=[e.render(req.source) for e in r.hard_errors])
    blob = midi_blob(r)
    stem = os.path.splitext(os.path.basename(req.name or "score"))[0] or "score"
    return Response(blob, media_type="audio/midi",
                    headers={"Content-Disposition": f'attachment; filename="{stem}.mid"'})
```

The piano. `keyboard.js` builds 88 keys from A0 to C8; white keys are flex items and black keys are positioned absolutely between them with the small offsets of a real keyboard. Velocity follows where a key is clicked, dragging across keys plays a glissando, the computer keyboard maps A to L and W to P onto an octave, Z and X shift the octave and Space is the sustain pedal. The keys of the compiled key signature carry a dot, the root a larger one, and scale degrees can be shown on the keys, which ties the interface to the inherited attribute that makes `^5` a pitch.

The sound. `audio.js` synthesises a piano in the Web Audio API: six partials with inharmonicity rising with pitch, a detuned second string, a filtered noise burst for the hammer, a low-pass filter whose cut-off follows velocity, a decay that is long in the bass and short in the treble, a convolution reverb from a generated impulse and a compressor. Other General MIDI programs the compiler can emit map to electric piano, harpsichord, bass, strings, pad and drum timbres, and a sampled grand can be loaded from a public soundfont. A look-ahead scheduler plays the compiler's timed events at sample-accurate times while the keys light up in each voice's colour and the playhead glides across the roll.

Listing 23. audio.js: the piano voice of the synthesiser

```
case 'piano':
case 'honky':
case 'bright': {
  const B = 0.00012 + spread * 0.0009;              // inharmonicity
  const partials = [[1, 1.0, 'triangle'], [2, 0.42 * bright, 'sine'], [3, 0.2 * bright, 'sine'],
```

```
                  [4, 0.09 * bright, 'sine'], [5, 0.045 * bright, 'sine'], [6, 0.02 * bright, 'sine']];
  for (const [n, amp, type] of partials) {
    const g = osc(type, f * n * Math.sqrt(1 + B * n * n), amp);
    g.gain.setTargetAtTime(0, t + 0.02, stringDecay / (1.6 + n * 0.9));
  }
  osc('triangle', f, 0.38, timbre === 'honky' ? 9 : 1.8);
  if (timbre === 'honky') osc('triangle', f, 0.3, -7);
  if (timbre === 'bright') cutoff *= 1.5;
  noise(2200 + 2600 * v, 0.9, 0.5 * v, 0.045);     // the hammer
  noise(90, 2, 0.12 * v, 0.06, 'lowpass');            // the body thump
  break;
}
```

Capture. With capture on, what is played is written as Cadence tokens: a note length is chosen from a palette, melody mode writes one token per key, chord mode gathers the keys held into `<...>/4`, and degree mode spells notes as `^1 ^3 ^5` of the compiled key by inverting the compiler's own degree formula. A palette of the diatonic chords of the key is built from the same theory module. The tokens can be inserted at the cursor or wrapped into a `phrase captured1 = [ ... ] ;` before the first voice.

Listing 24. theory.js: inverting the compiler's resolution of a scale degree

```
export function degreeOf(p, keyPc, mode) {
  const scale = SCALES[mode] || SCALES.major;
  for (let shift = -4; shift <= 4; shift++) {
    for (let i = 0; i < 7; i++) {
      if (60 + 12 * shift + keyPc + scale[i] === p) return { degree: i + 1, shift };
    }
  }
  return null;
}

export function degreeToken(p, keyPc, mode) {
  const d = degreeOf(p, keyPc, mode);
  if (!d) return null;
  return '^' + d.degree + (d.shift > 0 ? "'".repeat(d.shift) : ','.repeat(-d.shift));
}
```



Figure 42. *Playback: the playhead on the roll, the clock in the top bar, and the keys lit in the voice colours.*



Figure 43. *The capture strip while recording: notes played on the keyboard as tokens (G4 B4), then in degree spelling of the compiled key (^3 ^1), then a chord from the palette (<^1 ^3 ^5>/4).*



Figure 44. *The keyboard dock: labels, sound, octave, volume, room, the pedal indicator and the capture switch.*



Figure 45. *The ten mood presets for the piano family; each card shows its key, tempo, note count and length as compiled.*



Figure 46. *The Tempest preset: D minor at 160 bpm, driving sixteenths over hammered octaves.*



Figure 47. *The same workstation in the ivory theme.*

Table 14. *The mood presets, compiled by the Studio's own presets module; all compile with no warnings*

Preset	Mood	Key	Tempo	Notes	Length (s)	Voices
Nocturne	Melancholic	E minor	66	87	29.09	2
Morning Light	Joyful	G major	128	124	16.88	2
Rain on Glass	Contemplative	A minor	72	63	33.33	2
Tempest	Dramatic	D minor	160	179	12.0	2
Waltz in Ivory	Elegant	F major	150	140	19.2	2
Lullaby	Tender	C major	76	56	25.26	2
Invention	Scholarly	D minor	112	103	10.71	2
Pulse	Hypnotic	C major	120	144	16.0	2
Blue Hour	Jazzy	D dorian	96	98	20.0	2
Hymn	Solemn	G mixolydian	60	40	18.0	2

The composer. `ai.py` sends the Groq model a system prompt containing the complete language reference and the user's score together with what the compiler says about it, then compiles whatever program the model returns. If the compiler rejects it, the diagnostics are sent back and the model is asked to repair it, up to three attempts. The browser therefore never receives a program that does not compile without being told so; a green badge means the proposal compiled and reports its notes and length, and the user can replace the score, audition the proposal without touching the score, or insert it. The API key is pasted in the browser and sent only to the user's own Studio server.

Listing 25. ai.py: the compose-compile-repair loop

```
def compose(api_key: str | None, model: str | None, action: str, prompt: str,
            source: str, history: list[dict] | None = None, temperature=0.7) -> dict:
    """Ask the model, compile what it proposes, and loop on failure."""
    client = make_client(api_key)
    model = (model or DEFAULT_MODEL).strip()
    messages = build_messages(action, prompt, source, history)
    attempts = []
    reply = ""
    code = None
    verified = None
    for attempt in range(1, MAX_ATTEMPTS + 1):
        reply = chat_once(client, model, messages, temperature=temperature)
        code = extract_code(reply)
        if code is None:
            attempts.append({"attempt": attempt, "code": False, "ok": None})
            break                                   # prose answer: nothing to verify
        verified = verify_code(code)
        attempts.append({"attempt": attempt, "code": True, "ok": verified["ok"],
                         "errors": len([e for e in verified["errors"] if e["severity"] == "error"])})
        if verified["ok"] or attempt == MAX_ATTEMPTS:
            break
        feedback = "\n".join(
    ...
```

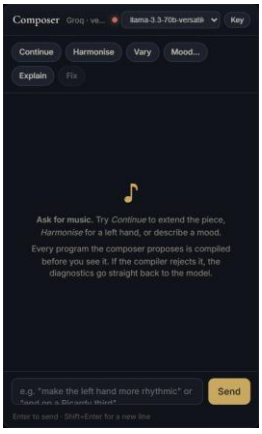


Figure 48. *The composer panel with its six actions; every proposal is compiled before it is shown.*

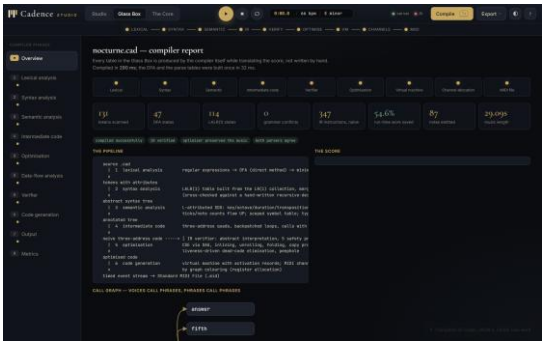


Figure 49. *The Glass Box overview: stage lights, metric cards, the status of both parsers, the verifier and the equivalence check, and the call graph.*

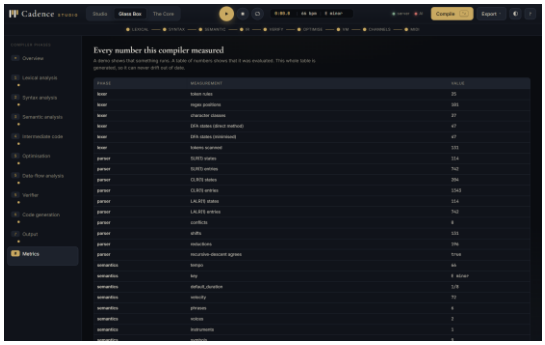


Figure 50. *Every number the compiler measured for the current score, generated so it can never drift out of date.*

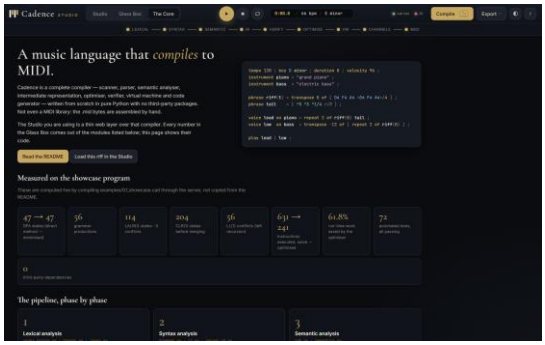


Figure 51. *The Core: the project statement and the measurements computed live from the showcase program.*

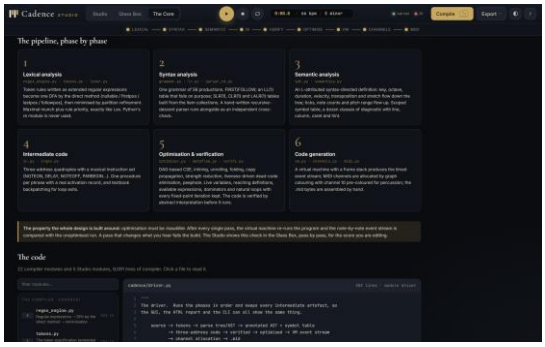


Figure 52. *The Core: the six phases of the pipeline with the files that implement each.*

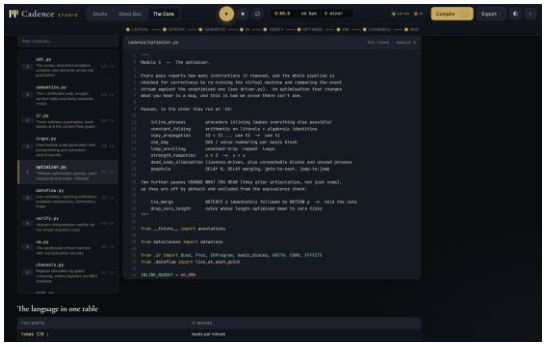


Figure 53. *The code browser inside the Studio, reading optimizer.py.*

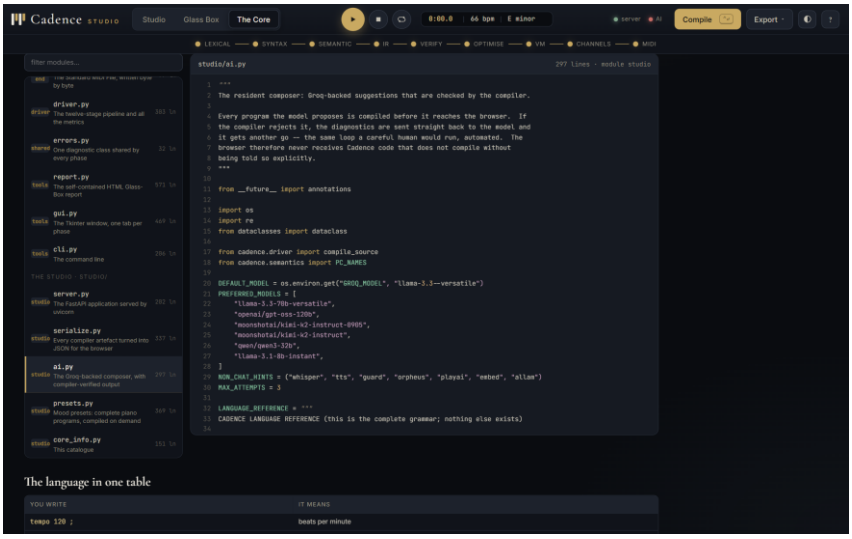


Figure 54. *The code browser reading the Studio's own composer module.*

5.11 Testing

The compiler ships 72 tests in `tests/test_all.py`, grouped by syllabus module, and the Studio adds 25 in `tests/test_studio.py`; all are plain `unittest`. The most important are the differential ones: the LALR and recursive-descent parsers must agree on every example; every example must compile at every optimisation level with an identical event stream; every individual pass must preserve the music; the verifier must reject each hand-crafted illegal program; the MIDI output must parse with an independent reader and have as many note-offs as note-ons; and the Studio's presets must compile with no warnings and end on the same bar in every voice. The composer is tested against a scripted stand-in model so no network is needed.

Table 15. *The test suite: 97 tests, all passing*

Group	What is defended	Tests
Module 1	direct method on (a|b)*abb, minimiser merges and preserves the language, character classes, maximal munch, priority, attributes, recovery	16
Module 2	FIRST/FOLLOW, left recursion, LALR conflict-free and as small as SLR, CLR larger, the ambiguous draft detected, both parsers agree, precedence, recovery	12
Module 3	degrees need the key, mode changes pitches, transposition accumulates, every class of semantic error, each broken example rejected in its phase, carets	15
Module 4	note pattern, backpatched loops, frames, chords	4
Module 5	music preserved at every level and after every pass, work reduced, dead code gone, folding, fixed points, loops found, verifier checks, sandbox	14
Module 6	percussion pre-coloured, simultaneous voices separated, spilling, sharing, MIDI bytes, determinism, tempo	8
End to end	reports generate, metrics present, compile time	3
Studio	presets, serializer, composer loop, every API endpoint	25

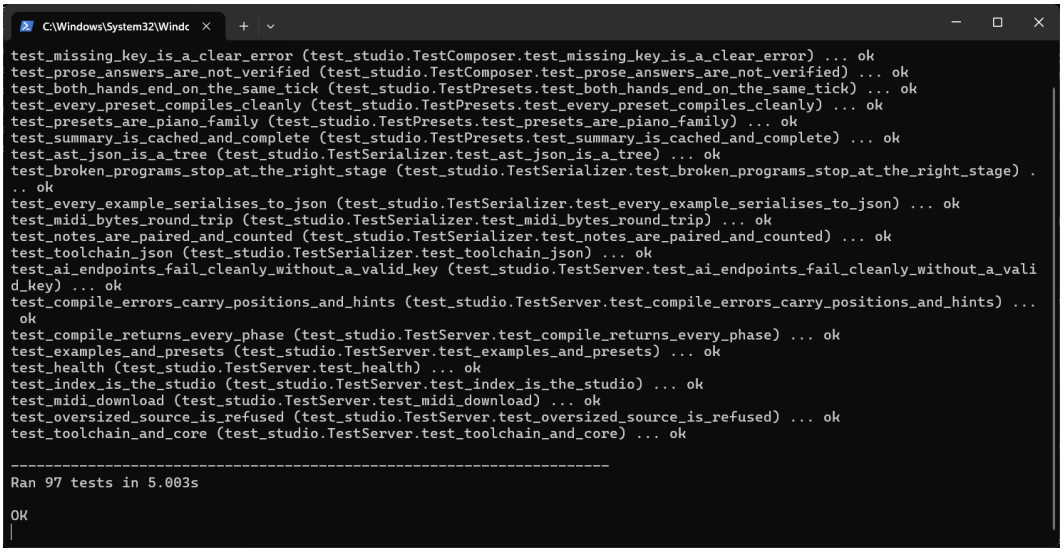


Figure 55. *The command line: python -m unittest discover -s tests -v, ending in Ran 97 tests, OK.*

5.12 Summary of results

Table 16. *Every number the compiler measured while compiling 07_showcase.cad*

Phase	Measurement	Value	Phase	Measurement	Value
lexer	token rules	25	semantics	voices	4
lexer	regex positions	101	semantics	instruments	4
lexer	character classes	37	semantics	symbols	14
lexer	DFA states (direct method)	47	semantics	warnings	0
lexer	DFA states (minimised)	47	ir	procedures (naive)	9
lexer	tokens scanned	168	ir	static instructions (naive)	173
parser	SLR(1) states	114	ir	procedures (optimised)	5
parser	SLR(1) entries	742	ir	static instructions (optimised)	247
parser	CLR(1) states	204	ir	static change	+42.8%
parser	CLR(1) entries	1343	runtime	instructions EXECUTED (naive)	631
parser	LALR(1) states	114	runtime	instructions EXECUTED (optimised)	241
parser	LALR(1) entries	742	runtime	work saved	61.8%
parser	conflicts	0	runtime	deepest activation-record stack	2
parser	shifts	168	runtime	note events	83
parser	reductions	197	runtime	length (ticks)	7680
parser	recursive-descent agrees	True	runtime	length (seconds)	9.6
semantics	tempo	100	runtime	optimiser preserved the music	True
semantics	key	E minor	backend	voices	4
semantics	default_duration	1/8	backend	channels used	4
semantics	velocity	92	backend	channel spills	0
semantics	phrases	4	backend	MIDI file size (bytes)	891

6. Conclusion:

Cadence meets the objectives set in Section 2. Every phase of the syllabus is implemented from its algorithm in pure Python, every intermediate artefact can be inspected in four different user interfaces, and the claims a compiler usually makes informally are measured: the parse tables are compared on the project's own grammar, the optimiser's effect is counted in executed instructions (61.8% less work on the showcase program), and its correctness is checked after every pass by re-running the program. The language itself turned out to be a good vehicle for the course because its features map onto the hard parts of compilation: scale degrees onto inherited attributes, repetition and transposition onto loops and arithmetic worth optimising, parallel voices onto register allocation with a fixed register file.

Cadence Studio extends the compiler into a usable instrument. It shows that a glass-box compiler can sit underneath a creative tool without changing a line of the compiler, and that the compiler's diagnostics are precise enough to steer a language model: the composer's proposals are accepted only when they compile.

6.1 Limitations

- There is no time signature; bars are whatever the programmer makes them, and the MIDI meta event always says 4/4.
- Articulation (ties, dynamics within a phrase) exists only as two optional passes that change the music and are therefore outside the equivalence check.
- The Studio's audio is a synthesis; the sampled grand needs the network. The composer needs a Groq API key.
- The grammar does not support nested phrase definitions or user-defined operators; a phrase may not call itself, so recursion is rejected rather than compiled.

6.2 Future work

- Static single-assignment form with phi-node insertion: dominance frontiers follow directly from the dominators already computed.
- A textual LLVM-IR back end, emitting a `.ll` file so that the same program can be compiled by an industrial back end.
- Real-time MIDI input in the Studio through the Web MIDI API, so that a physical keyboard captures to Cadence tokens.
- A time-signature declaration, and a `tie` operator so that articulation becomes part of the language rather than a pass.

7. Project Repository Link:

GitHub: https://github.com/rohitarora80905-glitch/CADENCE

The `main` branch holds the compiler, the Studio, the tests and the documentation; the `cadence-studio` branch carries the same work as a feature branch. The repository contains:

- `cadence/cadence/` the compiler (twenty modules, no dependencies) and `cadence/examples/` seven working programs plus three broken on purpose;
- `cadence/studio/` the FastAPI server, the serializer, the composer, the presets and the single-page Studio;
- `cadence/tests/` the test suite (`python -m unittest discover -s tests -v`);
- `cadence/docs/` the HTML report, the plain-English guide, the slide deck and the Studio how-to guide;
- `cadence/run.bat`, `cadence/run_studio.bat` and `cadence/build_all.bat` for Windows.

To run: `python -m cadence examples/07_showcase.cad --report r.html` compiles the showcase program and writes the report; `python -m cadence --gui` opens the Tkinter window; `pip install -r requirements-studio.txt` then `python -m studio` serves the Studio at http://127.0.0.1:8000/.